



- Message passing in SOA requires the use of two different protocol types: **the data interchange format and the network protocol that carries the message.**
- A client (or customer) connected to an **ESB communicates over a network protocol such as HTTP, Representational State Transfer (REST), or Java Message Service (JMS)** to a component (or service).
- Messages are most often in the form of the **eXtensible Markup Language (XML)** or in a variant such as the Simple Object Access Protocol (SOAP).
- SOAP is a messaging format used in **Web services that use XML as the message format** while relying on Application layer protocols such as HTTP and Remote Procedure Calls (RPC) for message negotiation and transmission.
- The software used to write clients and components can be written **in Java, .NET, Web Service Business Process Execution Language (WS-BPEL)**, or another form of executable code; the services that they message can be written in the same or another language.
- What is required is the **ability to transport and translate a message into a form that both parties can understand.**



- An ESB may require a variety of combinations in order to support communications between a service consumer and a service provider.
- For example, in WebSphere ESB, you might see the following combinations:
- XML/JMS (Java Message Service)
- SOAP/JMS
- SOAP/HTTP
- Text/JMS
- Bytes/JMS
- The Web Service Description Language (WSDL) is one of the most commonly used XML protocol for messaging in Web services, and it finds use in Service Oriented Architectures.



- Version 1.1 of WSDL is a W3C standard, but the current version WSDL 2.0 (formerly version 1.2) has yet to be ratified by the W3C.
- The significant difference between 1.1 and 2.0 is that version 2.0 has more support for RESTful (e.g. Web 2.0) application, but much less support in the current set of software development tools.
- The most common transport for WSDL is SOAP, and the WSDL file usually contains both XML data and an XML schema.
- REST offers some very different capabilities than SOAP. With REST, each URL is an object that you can query and manipulate.
- You use HTML commands such as GET, POST, PUT, and DELETE to work with REST objects. SOAP uses a different approach to working with Web data, exposing.
- Web objects through an API and transferring data using XML. The REST approach offers light weight access using standard HTTP command, is easier to implement than SOAP, and comes with less overhead.



- **SOAP is often more precise and provides a more error-free consumption model.** SOAP often comes with more sophisticated development tools.
- All major Web services use REST, but many Web services, especially newer ones, combine REST with SOAP to derive the benefits that both offer.

WSDL are essential objects to support message transfer, including these:

- The service object, a container where the service resides.
- The port or endpoint, which is the unique address of the service.
- The binding, which is the description of the interface (e.g. RPC) and the transport (e.g. SOAP).
- The portType, or interface that defines the capabilities of the Web service, and what operations are to be performed, as well as the messages that must be sent to support the operation.
- The operation that is to be performed on the message.



- The message content, which is the data and metadata that the service operation is performed on. Each message may consist of one or more parts, and each part must include typing information.
- The types used to describe the data, usually as part of the XML schema that accompanies the WSDL.
- A set of primers on WSDL may be found on the W3C Web site. The latest version, “Web Services Description Language (WSDL) Version 2.0 Part 0: Primer” may be found at <http://www.w3.org/2002/ws/desc/wsdl20-primer. n>

The code that follows is the WSDL 2.0 document that is created and analyzed in the W3C WSDL

- **primer referenced in the accompanying tip.**
- `<?xml version="1.0" encoding="utf-8" ?>`
- `<description`
- `xmlns="http://www.w3.org/ns/wsdl"`
- `targetNamespace= "http://greath.example.com/2004/wsdl/resSvc"`
- `xmlns:tns= "http://greath.example.com/2004/wsdl/resSvc"`
- `xmlns:ghns = "http://greath.example.com/2004/schemas/resSvc"`
- `xmlns:wsoap= "http://www.w3.org/ns/wsdl/soap"`
- `xmlns:soap="http://www.w3.org/2003/05/soap-envelope"`
- `xmlns:wsdIx= "http://www.w3.org/ns/wsdl-extensions">`



- `<documentation>`
- This document describes the GreatH Web service. Additional
- application-level requirements for use of this service --
- beyond what WSDL 2.0 is able to describe -- are available
- at <http://greath.example.com/2004/reservation-documentation.html>
- `</documentation>`
- `<types>` `<xs:schema`
- `xmlns:xs="http://www.w3.org/2001/XMLSchema"`
- `targetNamespace="http://greath.example.com/2004/schemas/`
- `resSvc"`
- `xmlns="http://greath.example.com/2004/schemas/resSvc">`
- `<xs:element name="checkAvailability"`
- `type="tCheckAvailability"/>`
- `<xs:complexType name="tCheckAvailability">`
- `<xs:sequence>`
- `<xs:element name="checkInDate" type="xs:date"/>`
- `<xs:element name="checkOutDate" type="xs:date"/>`
- `<xs:element name="roomType" type="xs:string"/>`
- `</xs:sequence>` `</xs:complexType>`



- `<xs:element name="checkAvailabilityResponse" type="xs:double"/>`
- `<xs:element name="invalidDataError" type="xs:string"/>`
- `</xs:schema>`
- `</types>`
- `<interface name = "reservationInterface" >`
- `<fault name = "invalidDataFault"`
- `element = "ghns:invalidDataError"/>`
- `<operation name="opCheckAvailability"`
- `pattern="http://www.w3.org/ns/wsdl/in-out"`
- `style="http://www.w3.org/ns/wsdl/style/iri"`
- `wsdlx:safe = "true">`
- `<input messageLabel="In"`
- `element="ghns:checkAvailability" />`
- `<output messageLabel="Out"`
- `element="ghns:checkAvailabilityResponse" />`
- `<outfault ref="tns:invalidDataFault" messageLabel="Out"/>`
- `</operation>`



- `</interface>`
- `<binding name="reservationSOAPBinding"`
- `interface="tns:reservationInterface"`
- `type="http://www.w3.org/ns/wsdl/soap"`
- `wsoap:protocol="http://www.w3.org/2003/05/soap/bindings/HTTP/">`
- `<fault ref="tns:invalidDataFault"`
- `wsoap:code="soap:Sender"/>`
- `<operation ref="tns:opCheckAvailability"`
- `wsoap:mep="http://www.w3.org/2003/05/soap/mep/soap-response"/>`
- `</binding>`
- `<service name="reservationService"`
- `interface="tns:reservationInterface">`
- `<endpoint name="reservationEndpoint"`
- `binding="tns:reservationSOAPBinding"`
- `address="http://greath.example.com/2004/`
- `reservation"/>`
- `</service>`
- `</description>`



- The XML document sets up the **namespace, defines the interface, specifies the binding, names the service, provides the documentations for the service**, and then supplies a schema that may be used to validate the document. In the message descriptions, the message types are declared.
- **XML schema can be separate files**, but what we see here is normal for WSDL, an inline schema that is part of the WSDL document. For a much more complete step-by-step description, refer to the W3C tutorial.
- A **WSDL file contains essential message data for a transaction**, but it **doesn't capture the full scope of a Service Oriented Architecture design**.
- Additional requirements need to be specified. The functional requirements for message passing between client and service in SOA are embodied in the **concept of a service contract**.
- A **service contract codifies the relationship between the data to be processed**, the metadata that accompanies that data, the intended service, and the manner in which the service will act upon that message.



- **Header:** The header contains the name of the service, service version, owner of the service, and perhaps a responsibility assignment. This is often defined in terms of a RACI matrix where the various roles and responsibilities for processes are spelled out in terms of a set of tasks or deliverables.
- The acronym designates the Responsible party or service, the Accountable decision maker, the Consulted party, and person(s) or service(s) that must be Informed on the use of the service.
- **Service Type:** Examples of service types include data, business, integration, presentation, and process types.
- **Functional Specification:** This category includes the functional requirements, what service operations or actions and methods must be performed, and the manner in which a service is invoked or initiated. Invocation usually includes the URL and the nature of the service interface.
- **Transaction attributes:** A message may define a transaction that may need to be managed or tracked or be part of or include another transaction operated at a specific Quality of Service and under a specific Service Level Agreement (SLA).
- Security parameters also are part of a transaction's attributes, as are the role the message plays in a process and the terms or semantics used to describe the interaction of the message with a service's interface.



Business Process Execution Language *Go, change the world®*

- If a message represents an **atomic transaction in a Service Oriented Architecture**, the next level of abstraction up is the grouping and managing of sets of transactions to form useful work and to execute a business process.
- An example of an execution language is the Business Process Execution Language (BPEL) or alternatively as the **Web Service Business Process Execution Language**.
- (WS-BPEL), a language standard for Web service interactions. The standard is maintained by the Organization for the Advancement of Structured Information Standards (OASIS) through their Web Services Business Process Execution Language Technical Committee.
- **BPEL is a meta-language comprised of two functions:** executable commands for Web services and clients, and internal or abstract code for executing the internal business logic that processes require.
- A meta-language is any language whose statements refer to statements in another language referred to as the object language.
- BPEL is often used to compose, orchestrate, and coordinate business processes with Web services in the SOA model, and it has commands to manage asynchronous communications.



Business Process Execution Language *Go, change the world®*

- BPEL uses XML with specific support for messaging protocols such as SOAP, WSDL, UDDI, WS-Reliable Messaging, WS-Addressing, WS-Coordination, and WS-Transactions.
- BPEL also builds on IBM's Web Services Flow Language (WSFL) and Microsoft's XLANG for data transport.
- The former is a system of directed graphs, while the latter is a block-structured language adding to additional verbs and nouns specific for business processes to BPEL, which were combined to form BPEL4WS and are being merged with BPEL.
- A version of BPEL to support human interaction is called BPEL4People, and it falls under the WS-HumanTask specifications of OASIS.
- BPEL was designed to interact with WSDL and define business processes using an XML language. BPEL does not have a graphical component.
- A business process has an internal or executable view and an external or abstract view in BPEL.



RV College of
Engineering®

Business Process Execution Language *Go, change the world®*

- One process may interact with other processes, but the goal is to minimize the number of specific extensions added to BPEL to support any particular business process.
- Data functions in BPEL support process data and control flow, manage process instances, provide for logic and branching structures, and allow for process orchestration.
- Because transactions are long-lived and asynchronous, BPEL includes techniques for error handling and scopes transactions. As much as possible, BPEL uses Web services for standards and to assemble and decompose processes.